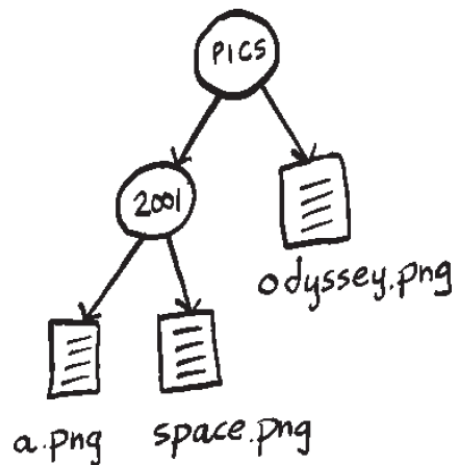


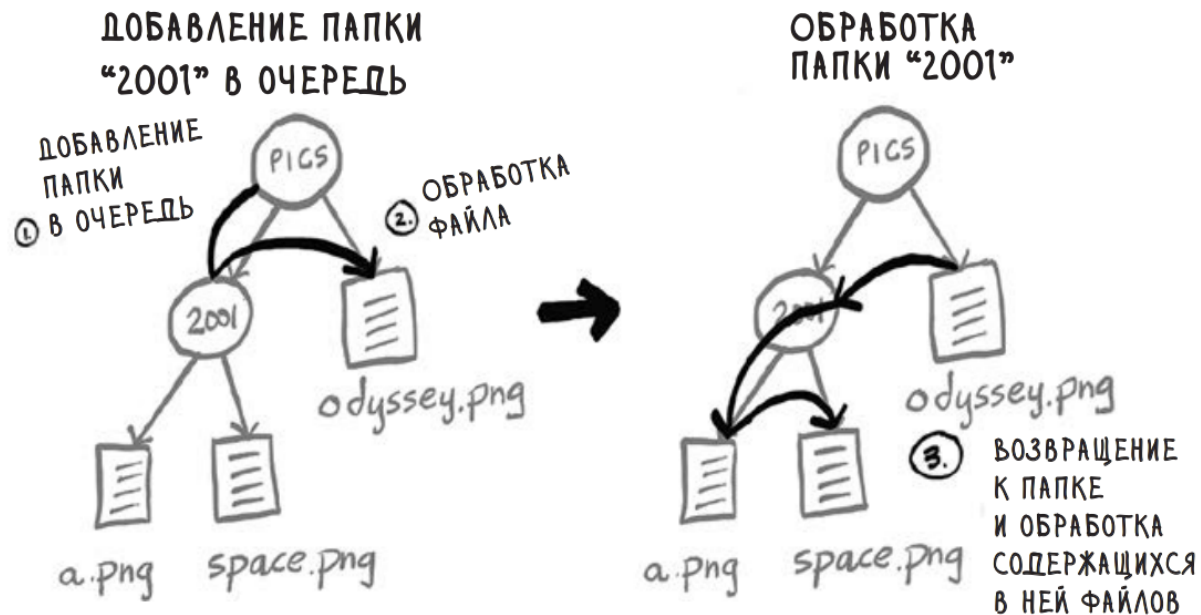
Поиск в глубину(DFS)

Пример

- 1. Посетить каждый узел в дереве.
- 2. Если узел является файлом, вывести его имя.

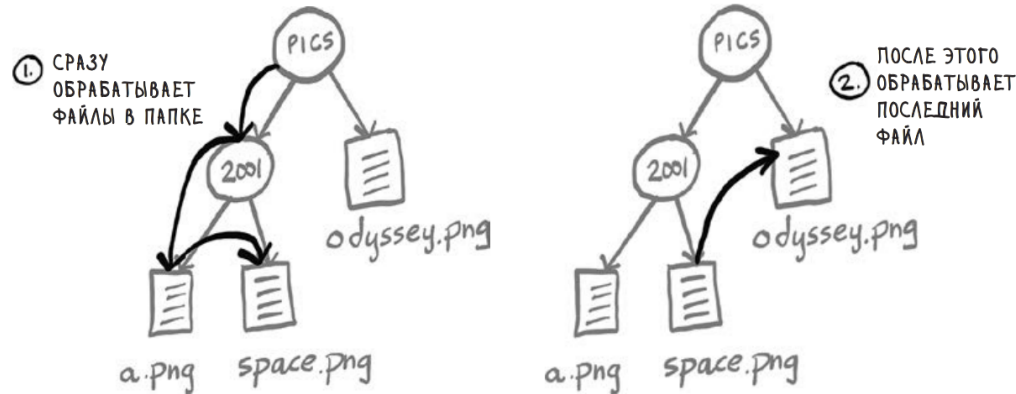


Поиск в ширину



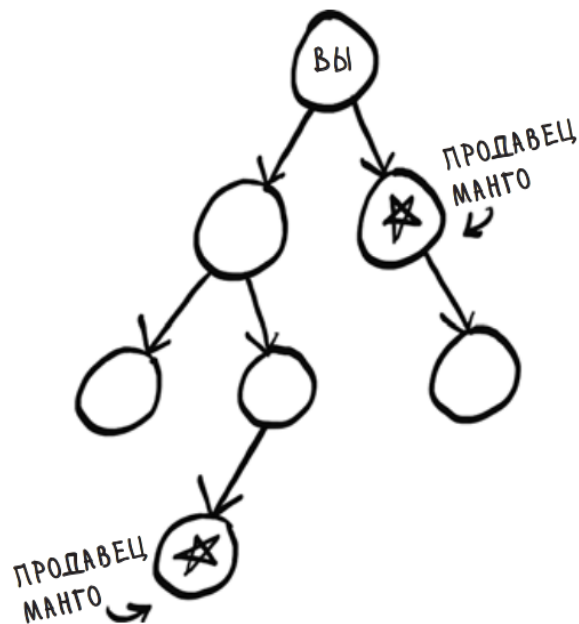
Поиск в глубину

- Поиск в глубину тоже работает с графом и представляет собой алгоритм обхода графа.
- При обнаружении папки он сразу заходит в нее, вместо того чтобы добавлять ее в очередь.



Применимость

- Поиск в ширину и поиск в глубину тесно связаны друг с другом, и там, где речь идет об одном из них, будет упоминаться и другой.
- Оба алгоритма выводят все имена файлов, так что они оба подойдут для нашего примера.
- Однако между ними существует важное различие: **Поиск в глубину не может использоваться для нахождения кратчайшего пути!**



Пример кода

```
from os import listdir
from os.path import isfile, join
```

```
def printnames(dir):
    for file in sorted(listdir(dir)):
        fullpath = join(dir, file)
        if isfile(fullpath):
            print(file)
        else:
            printnames(fullpath)
printnames("pics")
```

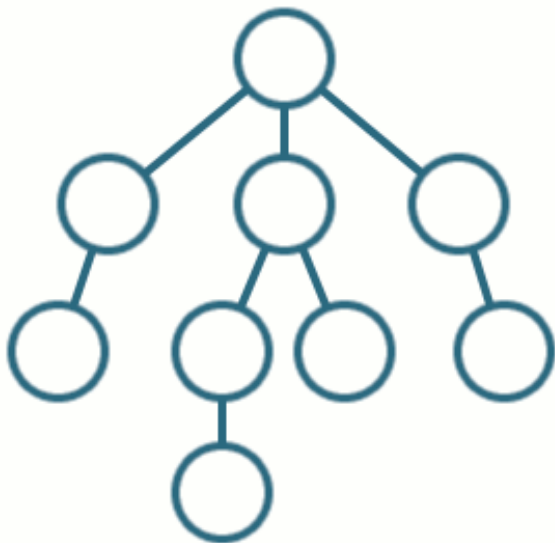
←..... Перебрать все файлы и все папки в текущей папке

←..... Если это файл, вывести его имя

←..... Если это папка, вызвать для нее функцию рекурсивно, чтобы провести поиск файлов и папок

Демонстрация

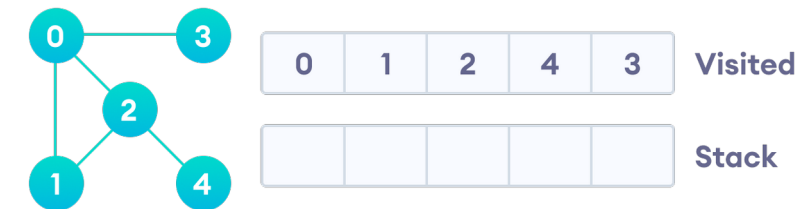
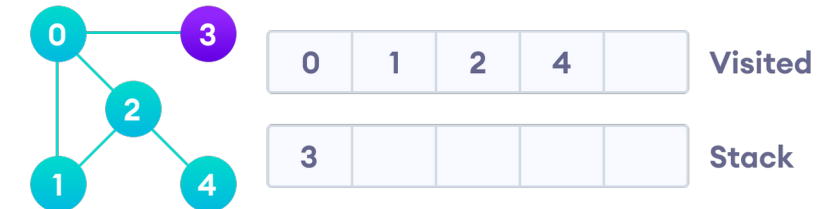
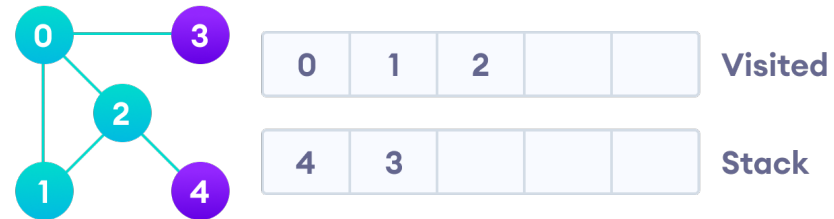
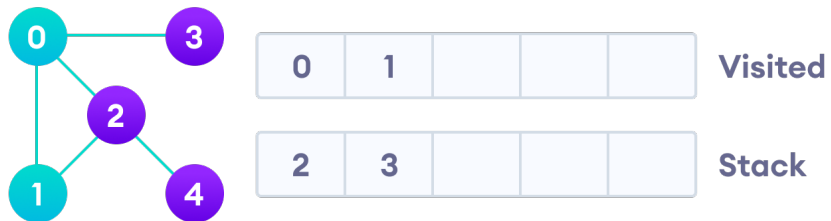
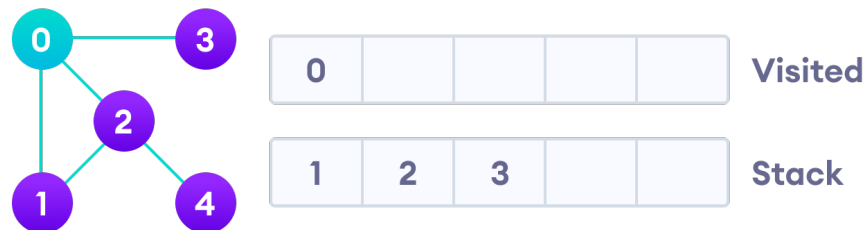
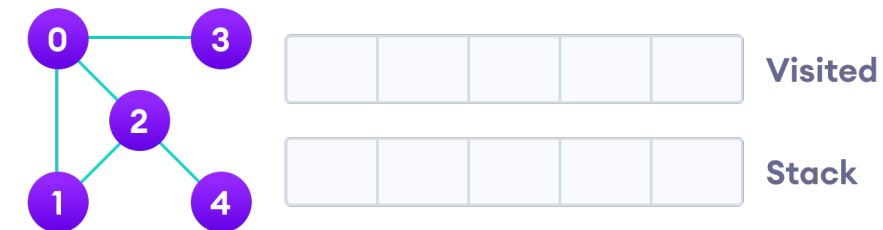
DFS



BFS



Пример со стеком



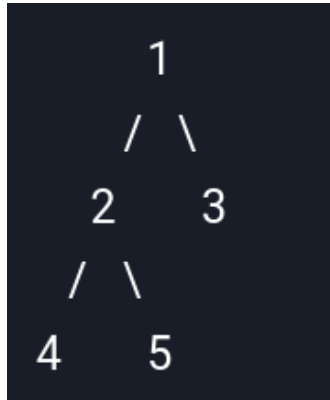
1. Обход дерева в глубину

- Условие:

Дано бинарное дерево. Выведите значения всех узлов в порядке DFS (префиксный обход: корень → левый → правый).

- Пример:

Вход:



- Выход: [1, 2, 4, 5, 3].

2. Сумма значений всех узлов

- Условие:

Найдите сумму всех значений узлов бинарного дерева.

- Пример:

Вход: Дерево из задачи 1.

- Выход: 15 ($1 + 2 + 3 + 4 + 5$).

3. Максимальная глубина дерева

- Условие:

Определите глубину бинарного дерева (максимальное расстояние от корня до листа).

- Пример:

Вход: Дерево из задачи 1.

- Выход: 3 (путь $1 \rightarrow 2 \rightarrow 4$ или $1 \rightarrow 2 \rightarrow 5$).

4. Зеркальное отражение дерева

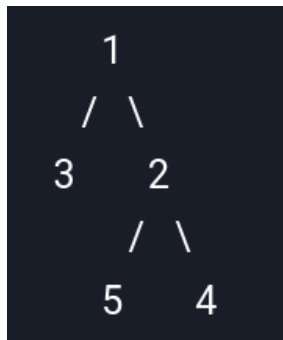
Условие:

Создайте зеркальное отражение бинарного дерева (поменяйте местами левые и правые поддеревья для каждого узла).

- Пример:

Вход: Дерево из задачи 1.

- Выход:



5. Поиск элемента в дереве

- Условие:

Проверьте, существует ли значение `target` в бинарном дереве.

- Пример:

Вход: Дерево из задачи 1, `target = 5`.

- Выход: `True`.

13. Поиск выхода в лабиринте

- Условие:

Дан лабиринт (матрица $N \times M$, 0 — проход, 1 — стена). Проверьте, существует ли путь от $(start_row, start_col)$ до (end_row, end_col) .

- Пример:

Вход:

```
maze = [  
    [0, 1, 0],  
    [0, 1, 0],  
    [0, 0, 0]  
]  
  
start = (0, 0), end = (2, 2)
```

- Выход: True.